YAJCP

Yet Another JSON Communication Protocol

Yet Another JSON Communication Protocol - **YAJCP** in short - is an application level communication protocol heavily inspired by HTTP developed to serve a very specific purpose: facilitate the message flow between client and server in the Java version of the game *My Shelfie*, developed by *Cranio Creations*.

Main interactions

This communication protocol aims to fulfil the main interactions that happen between client and server in the game My Shelfie:

- **Player authentication**: when a player wants to access a game or create a new one, they are asked to pick a username; if that username is not already taken, the player logs into the selected game with it. This enables the player to reconnect when network errors occur. The username is also shared with the other players in the game.
- **Player move communication**: the server asks the player that is currently playing for the move they would like to perform. The move is gathered by the game view, then packaged by the client-side controller and sent to the server for elaboration and validation; the server then responds with either a confirmation message or an error message, which will trigger the restart of the process.
- **View update**: since the game's model is stored in the server, the state of the game must be sent to every client after every player's move in order to display it through the view, which is a client-side component.
- **Game over notification**: once the game is over, every client must be sent a leaderboard that will announce the winner for that game.
- **Disconnection notification**: if a player is disconnected from the game, a notification is sent to other players.

Package composition

The basic building block of this communication protocol is the `Message` class.

A `Message` object is a serializable object that contains two fields:

- A `Header` object, which specifies the header of the message
- A `JSONArray` object, which contains the body of the message

The `Header` class is but an enumeration which contains all possible headers for the communication. Using an enumeration makes this communication protocol flexible, as adding new header types is very straightforward.

The message payload is contained in a `JSONArray`. Every element of the `JSONArray` is a `JSONObject` formatted in a specific way, depending on the header of the message. The body formats will be specified later in the documentation.

Headers

The message header specifies the purpose of the message and client-server communication is mainly based on the type of header received by the recipient.

Every header is identified by a *three-digit number*:

- The first digit denotes the *nature of the header*:

Digit	Description
1	Informational message
2	Confirmation message
3	Request message
4	Error message

- The second digit indicates the *scope of the header*:

Digit	Description
1	Login-related header
2	Game-related header

- The third digit *identifies* the specific header

A comprehensive list of headers can be found below:

Command	Code	Sender	Body	Description
`PING`	101	Server		This message is used to verify that the connection between client and server is still alive.
`GAME_OVER`	121	Server	`leaderboard: HashMap<String, Integer>`	This message contains a hash map with the leaderboard of the players to display to every user.
`GAME_UPDATE`	122	Server	`gameStatus: JSONObject`	Sent at the end of every turn in order to let the client update the player's view.
`OK`	200	Server/Client		An acknowledgement message that is sent when the contents of a previous response are valid.
`GAME_LIST_RESPONSE`	211	Server	`games: ArrayList<String>`	Once a player decides to join a game, the server responds with this message, which contains a list of all ongoing games.
`JOIN_GAME_RESPONSE`	212	Client	`gameId: String`	This message contains the game ID of the game the player wants to join. It is sent after the server provided the game ID list.
`SEND_TILES`	221	Client	`coordinates: JSONArray`	Sent in order to fulfil the `GET_TILES` request sent by the server.
`SEND_COLUMN`	222	Client	`column: int`	Sent in order to fulfil the `GET_COLUMN` request sent by the server.
`LOGIN_REQUEST`	311	Client	`username: String`	Sent to the server in order to log the player in.
`NEW_GAME_REQUEST`	312	Client	`playerNumber: int`	Sent when the user wants to create a new game.
`JOIN_GAME_REQUEST`	313	Client	`gameId: String`	Sent when the user wants to join an existing game.
`RECONNECT`	314	Client	`username: String`	Sent by the client when a disconnected player wants to reconnect to his game.
`GET_TILES`	321	Server		The user is required to select up to three tiles from the board.
`GET_COLUMN`	322	Server		The user is required to select which column they want the selected tiles to go into.
`BAD_HEADER`	400	Server		This message is sent when the server receives a message with an unexpected or wrong header.
`USERNAME_TAKEN`	411	Server		Sent to the client if the username that was chosen by the user is already in use.
`BAD_GAME_ID`	412	Server		Sent if the player that tried to join a game entered a wrong or non-existent game ID.
`NO_GAMES`	413	Server		Sent by the server when there are no available games to join.

Command	Code	Sender	Body	Description
`BAD_TILES`	421	Server		Sent if the game controller notices that the tiles sent by the player are not pickable.
`BAD_COLUMN`	422	Server		Sent if the game controller notices that the selected column is not suitable to hold the selected tiles.
`GAME_UNAVAILABLE`	423	Server		Sent by the server when a player tries to reconnect to a finished game.
`PLAYER_NOT_FOUND`	424	Server		Sent by the server if a player that never logged in tries to reconnect to a game.
`PLAYER_TERMINATED`	425	Server		Sent by the server when the client connection is severed by the server due to inactivity.
`PLAYER_DISCONNECTED`	426	Server	`username: String`	Sent by the server to notify all clients of a player disconnection.

Body formatting

The body of the message will be formatted differently according to what the message header is. Below, an exhaustive documentation of the body composition will be found for every message that has a non-empty body.

`GAME_OVER`

```
{
        "header": GAME_OVER,
        "body": [
                {
                        "username": "username",
                        "points": int
                },
                {
                        "username": "username",
                        "points": int
                }
        ]
}
```

Note that since the game is for 2-4 players, the number of `JSONObjects` inside the body of the message can vary between 2 and 4.

`GAME_UPDATE`

```
{
        "header": GAME_UPDATE,
        "body": [
                "board": {
                        "board": [
                                [/* Tile JSONArray x9 */]
                        ]
                },
                "players": [
                        {
                                "playerShelf": {
                                        "contents": [
                                                [/* JSONArray of JSONObject with field "value" (Tile)
*/]
                                        ]
                                },
                                "objectiveCard": {
                                        "code": int
```

```
                        },
                        "pointCards": [
                                {
                                        "value": int
                                },
                                {
                                        "value": int
                                }
                        ],
                        "endGameCard": boolean
                },
                {
                        "playerShelf": {
                                "contents": [
                                        [/* JSONArray of JSONObject with field "value" (Tile)
*/]
                                ]
                        },
                        "objectiveCard": {
                                "code": int
                        },
                        "pointCards": [
                                {
                                        "value": int
                                },
                                {
                                        "value": int
                                }
                        ],
                        "endGameCard": boolean
                }
        ],
        "pointDecks": [
                {
                        "size": int
                        "cards": [
                                {
                                        "size": int,
                                        "cards": [
                                                {
                                                        "value": int
                                                }
                                        ]
                                }
                        ]
                },
                {
                        "size": int
                        "cards": [
                                {
                                        "size": int,
                                        "cards": [
                                                {
                                                        "value": int
                                                }
                                        ]
                                }
                        ]
                }
        ],
        "objectives": [
                {
                        "cardType": String
                },
                {
                        "cardType": String
```

```
                        }
                ],
                "lastTurn": boolean
        ]
}
```

`GAME_LIST_RESPONSE`

```
{
        "header": GAME_LIST_RESPONSE,
        "body": [
                {
                        "games": ArrayList<String>
                }
        ]
}
```

`JOIN_GAME_RESPONSE`

```
{
        "header": JOIN_GAME_RESPONSE,
        "body" :[
                {
                        "gameId": String
                }
        ]
}
```

`SEND_TILES`

```
{
        "header": SEND_TILES,
        "body": [
                {
                        "row": int,
                        "column": int
                },
                {
                        "row": int,
                        "column": int
                },
                {
                        "row": int,
                        "column": int
                }
        ]
}
```

The message body does not need to contain exactly three tiles, but can contain between one and three *different* tiles.

`SEND_COLUMN`

```
{
        "header": SEND_COLUMN,
        "body": [
                {
                        "column": int
                }
        ]
}
```

`LOGIN_REQUEST`

```
{
        "header": LOGIN_REQUEST,
        "body": [
                {
                        "username": String
                }
        ]
}
```

`NEW_GAME_REQUEST`

```
{
        "header": NEW_GAME_REQUEST,
        "body": [
                {
                        "playerNumber": int
                }
        ]
}
```

`JOIN_GAME_REQUEST`

```
{
        "header": JOIN_GAME_REQUEST,
        "body": [
                {
                        "gameId": String
                }
        ]
}
```

`RECONNECT`

```
{
        "header": RECONNECT,
        "body": [
                {
                        "username": String
                }
        ]
}
```

`PLAYER_DISCONNECTED`

```
{
        "header": PLAYER_DISCONNECTED,
        "body": [
                {
                        "username": String
                }
        ]
}
```